

LieCFM

Lie 대수 기반 조합 Flow Matching · 설계 및 개발 문서 v2

PGFM 프로젝트 · 2026-08-11 · 데이터/split/평가 구축 완료, 모델 미착수

v1 과의 차이. 이 문서의 모든 수치는 레포의 실행 가능한 코드에서 재생성된다. 근거 코드가 없는 주장은 넣지 않았다. 각 절 끝에 산출 코드를 명시하고, 마지막 절에 수치-코드 대응표를 둔다.

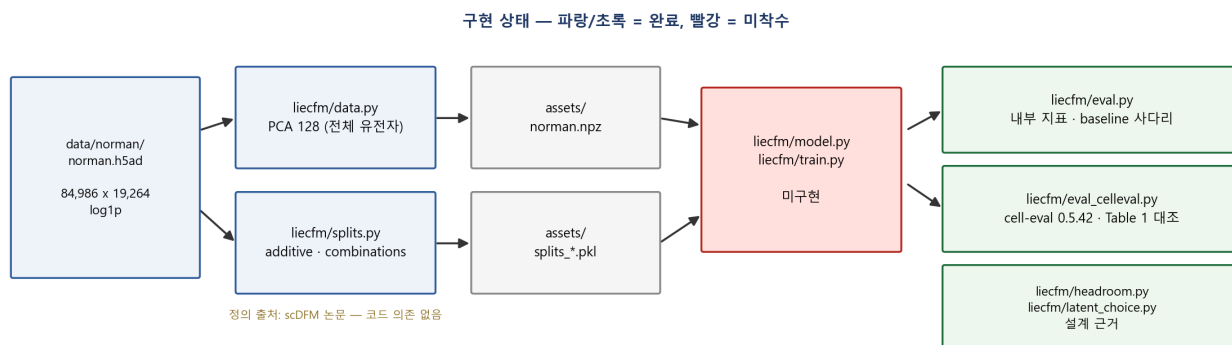


그림 1. 구현 상태. 박스는 이 저장소에서 실제로 실행되는 것만 표시했다. 선행 연구(scDFM)는 split 정의의 출처일 뿐 실행 구성요소가 아니므로 주석으로만 적었다.

1. 데이터 파이프라인

Norman h5ad(84,986 x 19,264, normalize_total + log1p 적용본)를 **전체 유전자**에 대해 PCA 128차원으로 사영한다. 누적 설명분산 0.131.

1.1 HVG 를 쓰지 않는 이유

평가 지표는 조건별 top-20 DE 유전자에서 계산된다. HVG 로 잘라내면 그 유전자들이 잠재 공간에 없어 지표 자체를 낼 수 없다. 실측 결과다.

기준	상위 k	DE20 합집합 포함률	조건별 커버리지 (중앙 / 최소)	섭동 표적 포함
원분산	2,000	38.1%	75% / 5%	19/101
원분산	5,000	60.1%	90% / 10%	39/101
원분산	8,000	81.7%	100% / 45%	66/101
dispersion z	2,000	23.9%	55% / 0%	40/101
dispersion z	5,000	45.1%	65% / 15%	63/101
dispersion z	8,000	56.1%	70% / 20%	77/101

dispersion 기준이 오히려 나쁜 이유는 DE20 이 non-dropout 기준이라 고발현 유전자 편향이기 때문이다. 원분산 8,000개를 써도 조건별 최소 커버리지가 45%다. 따라서 전체 유전자를 쓴다. 임의 선택과 잠재적 leakage 경로가 함께 사라진다.

산출: liecfm/latent_choice.py · 결과 results/latent_choice.json

1.2 구현 중 잡은 결함

sklearn `PCA.fit_transform` 은 `randomized SVD` 의 `U-S` 를 그대로 반환하는데, 이는 $(X - \text{mean}) @ W^T$ 와 근사적으로만 같다. 그대로 두면 잠재 Z 와 디코더 W 가 정합하지 않아 모델이 두 공간을 오갈 때마다 오차가 누적된다.

실측: 조건 `delta` 투영 상대오차 **1.56e-02** -> fit 후 transform 명시 호출로 **~1e-06**

산출: `liecfm/data.py` · 결과 `assets/norman.npz`

1.3 잠재 차원을 128 로 정한 근거

조건 `delta` 를 잠재 부분공간에 사영했을 때의 복원 오차를 노이즈 바닥과 비교했다. 복원 오차의 상당 부분은 애초에 측정 노이즈이므로, 노이즈를 뺀 성분만이 실제로 잃는 신호다.

잠재 차원	delta 복원오차 (중앙)	q90	노이즈 제외 실손실
32	0.4224	0.5965	0.299
64	0.3686	0.5354	0.217
128	0.3441	0.4991	0.172
256	0.3287	0.4829	0.138
512	0.3075	0.4592	0.075
노이즈 바닥	0.2983	0.4296	—

128차원에서 실제로 잃는 신호는 0.17 이고, 512차원으로 4배 늘려도 0.07 까지만 줄어든다. 비용 대비 이득이 없다.

더 중요한 확인: 학습 조건의 `delta` 로 만든 기저가 세포 PCA 보다 낮지 않다 (rank 128 에서 `delta` 기저 0.3233 vs 세포 PCA 0.3082, 노이즈 바닥 0.2724). 섭동이 세포가 원래 변이하는 축을 따라 움직이기 때문이다.

즉 남는 성분은 조건 고유 방향이라 다른 조건에서 배울 수 없고, 어떤 방법도 도달하지 못하는 상한이다. 잠재 차원 선택이 우리 모델만 불리하게 만들지 않는다. (기저는 leakage 를 피해 train 조건만으로 만들고 held-out 조건에서 평가했다.)

산출: `liecfm/latent_choice.py`

2. Split — 선행 연구 정의를 그대로 사용

자체 설계 split 은 '저자가 만든 split 에서 저자의 방법이 이겼다'는 반박에 답할 수 없다. 따라서 scDFM(arXiv:2602.07103) 의 정의를 난수 방식까지 그대로 이식했다.

split	정의	규모	재현성
additive	double 을 셔플해 30% 를 test. single 은 전부 train	fold 5개 · train 88 / test 37	원본 파일과 대조 검증
combinations	additive test 앞 15개만 test 로 두고 그 유전자의 single 도 test 로	test 15d + held-out single 21~26	additive 에서 결정적 파생

2.1 난수 재현성

scDFM 은 **legacy RandomState**(np.random.seed / shuffle)를 쓴다. NumPy 는 NEP 19 에서 RandomState 의 스트림을 동결했으므로 numpy 버전이 달라도 같은 split 이 나온다. (default_rng 계열은 이 보장이 없다 — 쓰지 않는다.)

liecfm/splits.py 는 매 실행마다 원본 산출물 **data/norman/split_results.pkl** 과 대조하고, 불일치하면 중단한다. 현재 5개 fold 의 train/test 가 모두 일치한다.

원본의 세 번째 방식 **unseen** 은 이식하지 않았다. (1) 시드 없는 random.shuffle 을 쓰고, (2) 원본 119행 **if split_method == 'additive' or 'combinations':** 이 항상 참이라 해당 분기가 도달 불가능한 죽은 코드다.

2.2 scDFM 에 대한 의존 관계

실행 의존성은 없다. 정의를 이 저장소의 코드로 옮긴 뒤로 scDFM 저장소를 실행하거나 임포트하는 곳이 없다. 코드 안의 scDFM 언급은 전부 **출처 표기**다.

유일한 파일 연결은 **data/norman/split_results.pkl** 인데, 이건 우리 데이터 폴더에 있는 **검증 앵커**다. 없어도 split 은 결정적으로 동일하게 생성되며 대조 검증만 건너뛰다.

산출: **liecfm/splits.py** · 결과 **assets/splits_additive.pkl**, **assets/splits_combinations.pkl**

3. 평가 — 두 층

3.1 내부 하네스 (모델 개발용)

edist_rel PC 잠재 공간 energy distance / control 대비. 0 = 완벽, 1 = control 과 동일

resid_R2 가법 잔차를 얼마나 맞추는가. 가법 예측은 정의상 0.

$r = m_{AB} - m_A - m_B + m_{ctrl}$

$r_{hat} = m_{hat} - m_A - m_B + m_{ctrl}$

$R2 = 1 - (\|r_{hat} - r\|^2 - E_{noise}) / (\|r\|^2 - E_{noise})$

r_de20 top-20 DE Pearson(delta). 포화되므로 보고만 하고 판정에 쓰지 않는다

산출: **liecfm/eval.py**

3.2 외부 비교 (논문 표 대조용)

scDFM Table 1 은 **cell-eval 0.5.42** 의 MetricsEvaluator 출력이다. 우리 숫자를 그 표와 직접 비교하려면 같은 패키지-같은 프로토콜이어야 한다. 평가 유전자는 adata_test(=test 조건 + control)에서 고른 HVG 1,000개, 조건당 예측 세포 128개, fold=1 이다.

Table 1 열	cell-eval 지표명	논문 Additive	우리 재현	상대차
MSE	mse	0.00448	0.004442	0.86%
MAE	mae	0.02276	0.022542	0.96%
Pearson Δ	pearson_delta	0.9024	0.898994	0.38%
DS	discrimination_score_l1	0.9686	0.975895	0.75%
DE-Spearman ρ	de_spearman_sig	0.5564	0.588215	5.72%

비교 가능 범위를 정확히 밝힌다.

- Table 1 의 8개 열 중 **5개만 cell-eval 0.5.42 에서 나온다**. L2, 두 번째 Pearson Δ , Pearson Δ_{20} 는 그 패키지의 27개 지표에 **없고**, scDFM 공개 저장소에도 구현이 없다 — 논문 표 작성 시 별도 후처리로 보인다. 정의를 확정하기 전까지 그 세 열은 **비교하지 않는다**.
- 위 재현은 **단일 시드 1회 실행**이다. 조건당 control 세포 128개를 무작위로 뽑으므로 표본 변동이 있다. 상위 3개 지표의 차이(<1%)가 그 변동 안인지는 **아직 측정하지 않았다**.
- DE-Spearman ρ 의 5.7% 차이는 더 크다. 이 지표는 예측 세포 전체에서 DE 를 다시 계산하므로 128개 표본 추출에 민감하다.

산출: `liecfm/eval_celleval.py` · 전용 환경 `.env-celleval` (python 3.11, cell-eval 0.5.42 가 python<3.13 요구)

4. 목표선 — 모델이 넘어야 할 숫자

split	baseline	r_de20	edist_rel ↓	resid_R2 ↑
additive	control	—	1.0000	-5.5985
additive	additive delta_A + delta_B	0.9770	0.2207	0.0000
additive	per-gene scaled	0.9795	0.1807	0.5313
additive	ridge additive	0.9846	0.1980	0.7201
combinations	가법 계열 4종	—	계산 불가	계산 불가
combinations	ridge additive	0.9808	0.2494	0.4092

LieCFM 이 넘어야 할 선

additive edist_rel < **0.1807** · resid_R2 > **0.7201**

combinations edist_rel < **0.2494** · resid_R2 > **0.4092**

combinations 에서 가법 baseline 이 계산 불가한 이유는 test double 의 single 이 학습에서 제외되기 때문이다. 하네스가 자동으로 감지해 건너뛰었다.

5. 헤드룸 — 설계 근거

지금까지의 baseline 은 전부 control 구름의 **평행이동**이다 (예측세포 = control 세포 + delta). 따라서 분포의 모양은 control 그대로다. 남은 오차가 평균 때문인지 모양 때문인지 갈라야 모델의 표현력을 어디에 쓸지 정할 수 있다.

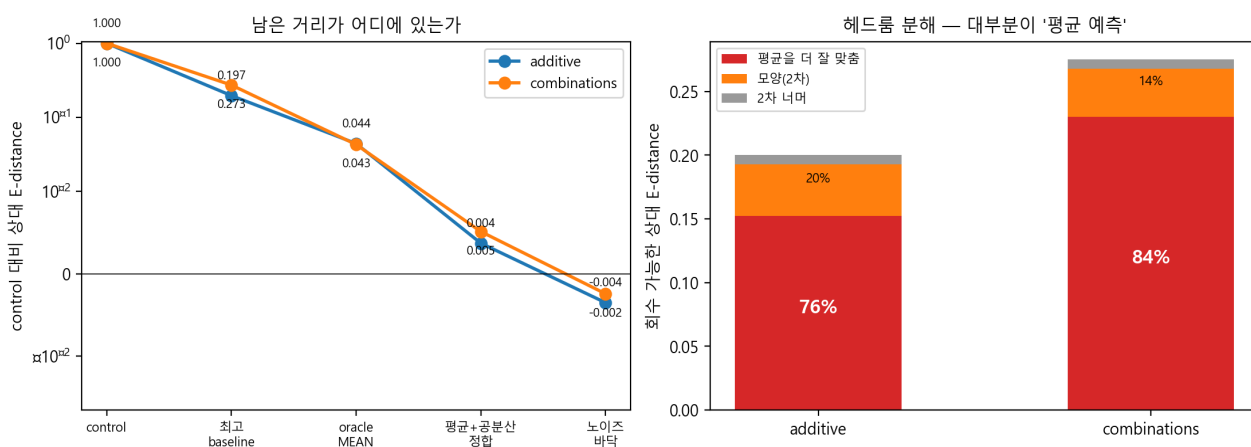


그림 2. oracle_mean = 참 delta 로 평행이동(평행이동 모델의 하한), gauss_match = 평균+공분산 정합, self_split = 노이즈 바닥.

split	최고 baseline	oracle_mean	평균 여지	모양(2차) 여지	2차 너머
additive	0.1966	0.0443	+0.1524 (76%)	+0.0406 (20%)	+0.0072
combinations	0.2729	0.0429	+0.2300 (84%)	+0.0378 (14%)	+0.0076

회수 가능한 거리의 대부분이 '평균 예측'이다 (additive 76%, combinations 84%). 그리고 평균+공분산만 맞추면 노이즈 바닥에 닿는다 (gauss_match 0.0037 vs self_split -0.0035).

설계 귀결 두 가지. (1) 생성원을 **affine** 으로 제한해도 표현력 손실이 거의 없다 — 남은 용량을 조합 구조에 배분한다. (2) 고차 분포 모델링은 이 데이터에서 보상받지 못하므로 거기에 파라미터를 쓰지 않는다.

산출: [liecfm/headroom.py](#)

6. 모델 — LieCFM

additivity = commutativity | epistasis = non-commutativity

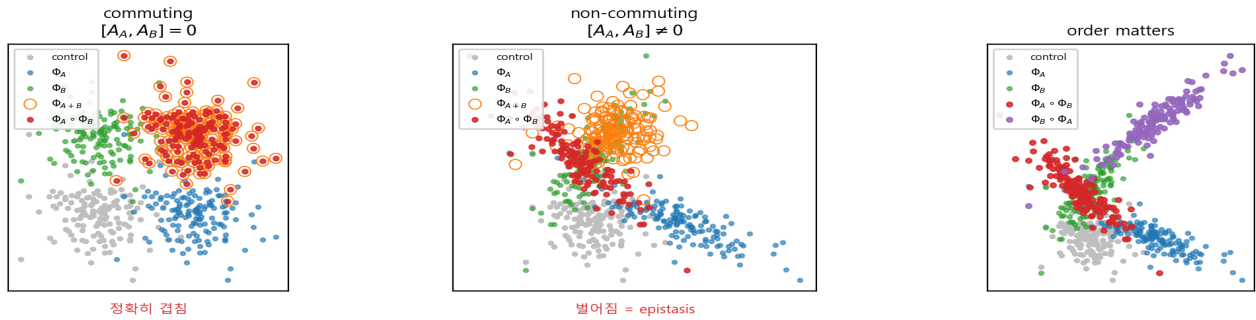


그림 3. 흐름의 합성이 가법적이지 않은 이유.

6.1 원리

flow matching 에서 각 섭동은 하나의 **속도장**을 유도한다. 속도장은 Lie bracket 아래 Lie 대수를 이루고, 흐름의 합성은 BCH 공식이 지배한다.

$$\Phi_A \circ \Phi_B = \Phi_{\{A + B + \frac{1}{2}[A, B] + (1/12)([A, [A, B]] - [B, [A, B]]) + \dots\}}$$

가법성 = 가환성. Epistasis = 비가환성.

6.2 구조

잠재 $z \in \mathbb{R}^{128}$ (전체 유전자 PCA. 역변환이 정확)

생성원 $u_a(z, t) = U_a(t) (V_a(t)^T z) + \beta_a(t)$ $A_a = U_a V_a^T$, rank r

교환자 $[u_a, u_b](z, t) = [A_b, A_a] z + A_b \beta_a - A_a \beta_b \leftarrow$ 학습 파라미터 없음

저랭크 계산 $A_b A_a z = U_b (V_b^T U_a) (V_a^T z)$, 비용 $O(d \cdot r + r^2)$

게이트 $\Lambda(a, b, t) = w(e_a, t) \odot m(e_b, t) - w(e_b, t) \odot m(e_a, t)$ (반대칭)

속도장 $v_\theta(z, t, P) = \sum_{a \in P} u_a(z, t) + \sum_{a < b \in P} \Lambda(a, b, t) \odot [u_a, u_b](z, t)$

교환자 항에 학습 파라미터가 없다. u_a 와 u_b 가 정해지면 두 섭동의 상호작용이 결정된다. 그래서 학습 때 본 적 없는 쌍의 상호작용을 구성 single 의 속도장만으로 계산할 수 있다. combinations split 에서 기존 방법과 갈리는 지점이다.

bracket 은 반대칭이고 게이트도 반대칭이므로 곱은 **a↔b 교환에 대칭**이 되고, $\Lambda(a, a) = 0$ 이라 자기 상호작용이 자동 소거된다. $P = \{ \}$ 이면 $v = 0$ — control 은 움직이지 않는다.

6.3 왜 flow matching 인가

근거	내용
구조적	Lie bracket 은 속도장 의 성질이다. delta 를 직접 회귀하는 모델에는 bracket 을 취할 대상 자체가 없다. 상호작용 항이 ODE 형식 없이는 존재할 수 없다
데이터	control 과 perturbed 세포는 짜지어져 있지 않다 . 세포 단위로 delta 를 회귀할 수 없고, FM + OT 가 짝을 만드는 방법이다

7. 구현 순서

단계	내용	판정 기준	상태
0	데이터 · split · 평가 하네스 · 헤드룸	목표선 확정	완료
1	잠재 affine FM 코어 (교환자 없음)	가법 재현 (edist_rel $\approx$ 0.221)	미착수
2	교환자 항 추가, on/off 비교	combinations 에서 edist_rel < 0.249	미착수
3	게이트 희소성 + 스펙트럴 정규화	목표선 안정 통과	미착수
4	자유 MLP Δ 대조군	대수적 형태의 기여 분리	미착수
5	cell-eval 로 Table 1 대조	scDFM 대비 위치 확정	미착수
6	전체 ablation	논문 표	미착수

2단계가 설계의 생사다. 교환자 항이 목표선을 넘지 못하면 구조를 바꾼다. 여기까지 만나절이면 판정된다.

8. 수치 - 코드 대응표

이 문서의 모든 수치는 아래 코드를 실행해 재생성된다.

절	수치	산출 코드	결과 파일
1	PCA 차원 · 설명분산 · 정합오차	liecfm/data.py	assets/norman.npz
1.1, 1.3	HVG 커버리지 · 잠재 차원 · 기저 비교	liecfm/latent_choice.py	results/latent_choice.json
2	split 규모 · 원본 일치 여부	liecfm/splits.py	assets/splits_*.pkl
3.2	cell-eval Additive 재현	liecfm/eval_celleval.py	results/celleval/
4	목표선 (baseline 사다리)	liecfm/eval.py	results/baseline_*.csv
5	헤드룸 분해	liecfm/headroom.py	results/headroom.csv

전체 재생성: `python -m liecfm.data ; python -m liecfm.splits ; python -m liecfm.eval ; python -m liecfm.headroom ;`

`python -m liecfm.latent_choice`

cell-eval 대조는 전용 환경에서: `.env-celleval/python.exe -m liecfm.eval_celleval`

이 PDF 는 위 결과 파일을 읽어 생성된다: `python docs/make_design_pdf.py`

남은 정확성 부채 (논문 전 해소 필요)

- ① cell-eval 재현이 단일 시드 1회다. 시드 반복으로 표본 변동 폭을 재야 <1% 차이를 '일치'라 부를 수 있다.
- ② Table 1 의 L2 · 두 번째 Pearson Δ · Pearson Δ_{20} 는 정의 미확보. Additive 행으로 정의를 역추적해 검증해야 8개 열 전부를 비교할 수 있다.
- ③ Combosciplex 는 25개 double 중 7개만 두 single 을 보유해 조합 평가에 쓸 수 없다. 이 문서는 Norman 만 다룬다.